Лабораторная работа №3. Markdown

Дисциплина: Архитектура компьютеров и операционные системы

Смирнов Артём Сергеевич

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **9**

4.1 Заполнение YAML-шапки 9
4.2 Написание цели, задания и теоретического введения 10
4.3 Написание раздела выполнения работы 11
4.4 Написание контрольных вопросов 12
4.5 Написание выводов 13
4.6 Компиляция отчёта 13
4.7 Проверка результатов 14

5 Выводы **15**

Список литературы **16**

Список иллюстраций

4.1 Заполнение YAML-шапки документа 10
4.2 Написание цели, задания и теоретического введения 11
4.3 Написание раздела выполнения лабораторной работы 12
4.4 Написание ответов на контрольные вопросы 13
4.5 Написание выводов и списка литературы 13
4.6 Запуск команды make для компиляции отчёта 14
4.7 Проверка содержимого каталога командой ls 14

Список таблиц

3.1 Основные элементы синтаксиса Markdown 7

1 Цель работы

Научиться оформлять отчёты с помощью легковесного языка разметки Markdown.

2 Задание

1. Сделать отчёт по предыдущей лабораторной работе в формате Markdown.
2. В качестве отчёта предоставить отчёты в 3 форматах: pdf, docx и md (в архиве, поскольку он должен содержать скриншоты, Makefile и т.д.)

3 Теоретическое введение

Markdown — легковесный язык разметки, созданный для обозначения форматирования в простом тексте с максимальным сохранением его читаемости. Основные элементы синтаксиса Markdown представлены в таблице 3.1.

Таблица 3.1: Основные элементы синтаксиса Markdown

Элемент	Синтаксис	Описание
Заголовки	`# Заголовок`	Знак # определяет уровень заголовка (1-6)
Полужирный	`**текст**`	Двойные звёздочки для полужирного начертания
Курсив	`*текст*`	Одинарные звёздочки для курсива
Списки	`- элемент`	Маркированный список с помощью тире
Нумерация	`1. элемент`	Нумерованный список с помощью цифр
Ссылки	`[текст](url)`	Встроенная гиперссылка
Изображения	`![подпись](путь)`	Вставка изображения
Код	`` `код` ``	Встроенный код в обратных кавычках
Блок кода	```` ``` ````	Огражденный блок кода

Элемент	Синтаксис	Описание
Формулы	`$формула$`	LaTeX формулы

Для обработки файлов в формате Markdown используется Pandoc (https://pandoc.org/). Pandoc позволяет конвертировать Markdown в различные форматы, включая PDF и DOCX.

Преобразование файла выполняется командами:

```
pandoc README.md -o README.pdf
pandoc README.md -o README.docx
```

Для автоматизации сборки используется Makefile.

4 Выполнение лабораторной работы

Отчёт по лабораторной работе №2 я изначально писал с использованием языка разметки Markdown, но через quarto, я немного изменил отчет добавил новый Makefile. Поэтому в рамках данной лабораторной работы описываю процесс создания этого отчёта.

4.1 Заполнение YAML-шапки

Открываю файл report.md в текстовом редакторе. Заполняю YAML-шапку документа: указываю название лабораторной работы (title), дисциплину (subtitle), автора (author), а также настройки форматирования для PDF-вывода (рис. 4.1).

Открыть

```yaml
---
## Front matter
title: "Лабораторная работа №2. Первоначальная настройка git"
subtitle: "Дисциплина: Архитектура компьютеров и операционные системы"
author: "Смирнов Артём Сергеевич"

## Generic otions
lang: ru-RU
toc-title: "Содержание"

## Bibliography
bibliography: bib/cite.bib
csl: pandoc/csl/gost-r-7-0-5-2008-numeric.csl

## Pdf output format
toc: true
toc-depth: 2
lof: true
lot: true
fontsize: 12pt
linestretch: 1.5
papersize: a4
documentclass: scrreprt
## I18n polyglossia
polyglossia-lang:
  name: russian
  options:
    - spelling=modern
    - babelshorthands=true
polyglossia-otherlangs:
  name: english
## I18n babel
babel-lang: russian
babel-otherlangs: english
## Fonts
mainfont: IBM Plex Serif
romanfont: IBM Plex Serif
sansfont: IBM Plex Sans
monofont: IBM Plex Mono
mathfont: STIX Two Math
mainfontoptions: Ligatures=Common,Ligatures=TeX,Scale=0.94
romanfontoptions: Ligatures=Common,Ligatures=TeX,Scale=0.94
sansfontoptions: Ligatures=Common,Ligatures=TeX,Scale=MatchLowercase,Scale=0.94
monofontoptions: Scale=MatchLowercase,Scale=0.94,FakeStretch=0.9
```

Рис. 4.1: Заполнение YAML-шапки документа

4.2 Написание цели, задания и теоретического введения

Формирую основные разделы отчёта: цель работы, задание и теоретическое введение. В теоретическом введении описываю основы работы с системами контроля версий и добавляю таблицу (рис. 4.2).

```
# Цель работы

Изучить идеологию и применение средств контроля версий. Освоить умения по работе с git.

# Задание

- Создать базовую конфигурацию для работы с git
- Создать ключ SSH
- Создать ключ PGP
- Настроить подписи git
- Зарегистрироваться на Github
- Создать локальный каталог для выполнения заданий по предмету

# Теоретическое введение

Системы контроля версий (Version Control System, VCS) применяется при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется.

Git — распределённая система контроля версий. В отличие от классических систем, в распределённых VCS центральный репозиторий не является обязательным — каждый разработчик имеет полную копию репозитория локально.

Основные команды git представлены в таблице [~#tbl:git-commands].

: Основные команды git {#tbl:git-commands}

| Команда | Описание |
|---------|---------|
| `git init` | Инициализация репозитория |
| `git clone <url>` | Клонирование репозитория |
| `git add .` | Добавить все изменения в индекс |
| `git commit -m "msg"` | Зафиксировать изменения |
| `git push` | Отправить на сервер |
| `git pull` | Получить с сервера |
| `git status` | Статус рабочего каталога |
| `git log` | История коммитов |
| `git branch` | Список веток |
| `git checkout -b branch` | Создать и переключиться на ветку |
| `git merge branch` | Слить ветку |
```

Рис. 4.2: Написание цели, задания и теоретического введения

4.3 Написание раздела выполнения работы

Описываю процесс выполнения лабораторной работы, добавляя текстовые описания выполненных действий, блоки кода с командами и ссылки на скриншоты. Для вставки изображений использую синтаксис Markdown с указанием идентификатора для перекрёстных ссылок (рис. 4.3).

```
# Выполнение лабораторной работы

![Установка git](image/1.png){#fig-001 width=70%}

![Установка gh](image/2.png){#fig-002 width=70%}

![Введение имя пользователя и почты владельца репозитория](image/3.png){#fig-003 width=70%}

![Настройка вывода сообщения формата utf-8](image/4.png){#fig-004 width=70%}

![Наименование ветки именем master](image/6.png){#fig-005 width=70%}

![Настраиваем параметры autocrlf и  safecrlf](image/7.png){#fig-006 width=70%}

![Создание SSH ключа размерностью 4096 бит](image/8.png){#fig-007 width=70%}

![Создание SSH ключа](image/9.png){#fig-008 width=70%}

![Создание PGP ключа](image/10.png){#fig-009 width=70%}

![Настройка PGP ключа (Выбор типа ключа, размерности ключа и срок его действия)](image/11.png){#fig-010 width=70%}

![Ввод личных данных(Имя и почта)](image/12.png){#fig-001 width=70%}

![Соглашение на корректность заполненной информации о ключе PGP](image/13.png){#fig-0011 width=70%}

![Вводим пароль, для ввода личных данных](image/14.png){#fig-0012 width=70%}

![Добавление личной информации в профиль на GitHub](image/git.png){#fig-0013 width=70%}

![Узнаем отпечаток PGP ключа и копируем ключ PGP в буфер обмен](image/15.png){#fig-0014 width=70%}

![Настройка автоматических подписей коммитов git](image/16.png){#fig-0015 width=70%}

![Настройка gh](image/17.png){#fig-0016 width=70%}

![Создаем директорию "Операционные системы" и переход в этот каталог](image/18.png){#fig-0017 width=70%}

![Выполняем создание копии репозитория на основе шаблона](image/19.png){#fig-0018 width=70%}

![Клонируем репозитории в созданную директорию в папку os-intro](image/20.png){#fig-0019 width=70%}

![Переход в каталог os-intro, удаляем лишний файл и создаем файл COURSE](image/21.png){#fig-00120 width=70%}
```

Рис. 4.3: Написание раздела выполнения лабораторной работы

4.4 Написание контрольных вопросов

Формирую раздел с ответами на контрольные вопросы. Использую нумерованный список для вопросов и форматирование Markdown для выделения ключевых терминов и блоков кода (рис. 4.4).

```
# Ответы на контрольные вопросы

**1. Что такое системы контроля версий (VCS)?**

VCS — программный инструмент для отслеживания изменений в файлах проекта. Позволяет фиксировать изменения, возвращаться к предыдущим версиям, работать нескольким разработчикам над одним проектом без конфликтов.

**2. Хранилище, commit, история, рабочая копия:**

- **Хранилище (репозиторий)** — место где хранятся все версии файлов и история изменений
- **Commit** — зафиксированный снимок состояния файлов в определённый момент с описанием изменений
- **История** — последовательность всех коммитов с указанием автора, даты и описания
- **Рабочая копия** — текущее состояние файлов на локальном компьютере, с которым работает разработчик

**3. Централизованные и децентрализованные VCS:**

- **Централизованные** — единый сервер хранит все версии, разработчики получает рабочую копию с него. Примеры: CVS, Subversion (SVN)
- **Децентрализованные** — каждый разработчик имеет полную копию репозитория локально. Примеры: Git, Mercurial

**4. Единоличная работа с VCS:**

```bash
git init        # создать репозиторий
git add .       # добавить файлы
git commit -m "message"  # зафиксировать изменения
git log         # просмотреть историю
```

5. Работа с общим хранилищем:

```bash
git pull        # получить изменения с сервера
# внести изменения в файлы
git add .
git commit -m "message"
git push        # отправить изменения на сервер
```

6. Основные задачи git:

Отслеживание изменений файлов, ведение истории версий, работа с ветками, слияние изменений от разных разработчиков, откат к предыдущим версиям.

7. Команды git:
```

Рис. 4.4: Написание ответов на контрольные вопросы

4.5 Написание выводов

Завершаю отчёт разделом выводов, в котором подвожу итоги выполненной работы, и добавляю раздел для списка литературы (рис. 4.5).

```
# Выводы

В ходе выполнения лабораторной работы изучил идеологию и применение средств контроля версий. Освоил базовую настройку git, создание SSH и PGP ключей, настройку подписей коммитов. Настроил GitHub CLI (gh) для удобной работы с GitHub из командной строки. Создал репозиторий курса на основе шаблона и настроил его для дальнейшей работы.

# Список литературы{.unnumbered}

::: {#refs}
:::
```

Рис. 4.5: Написание выводов и списка литературы

4.6 Компиляция отчёта

Выполняю компиляцию отчёта с помощью команды make. Pandoc обрабатывает файл report.md и создаёт новые файлы формата pdf и docx (рис. 4.6).

```
make
```

```
artemsmirnovsergeevich@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab02/repor
t$ make
pandoc "report.md" --filter pandoc-crossref --number-sections --citeproc -o "report.docx"
WARNING: pandoc-crossref was compiled with pandoc 3.6 but is being run through 3.6.4. This is not s
upported. Strange things may (and likely will) happen silently.
pandoc "report.md" --filter pandoc-crossref --pdf-engine=lualatex --pdf-engine-opt=--shell-escape
--citeproc  --number-sections -o "report.pdf"
WARNING: pandoc-crossref was compiled with pandoc 3.6 but is being run through 3.6.4. This is not s
upported. Strange things may (and likely will) happen silently.
[WARNING] [makePDF] LaTeX Warning: Label `fig-001' multiply defined.
[WARNING] [makePDF] LaTeX Warning: Empty bibliography on input line 549.
[WARNING] [makePDF] LaTeX Warning: There were multiply-defined labels.
artemsmirnovsergeevich@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab02/repor
t$
```

Рис. 4.6: Запуск команды make для компиляции отчёта

4.7 Проверка результатов

Проверяю содержимое каталога с помощью команды ls. В каталоге присутствуют все необходимые файлы(рис. 4.7).

```
ls
```

```
artemsmirnovsergeevich@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab02/repor
t$ ls
bib  image  Makefile  pandoc  report.docx  report.md  report.pdf  _resources
```

Рис. 4.7: Проверка содержимого каталога командой ls

5 Выводы

В ходе выполнения лабораторной работы научился оформлять отчёты с помощью легковесного языка разметки Markdown. Освоил синтаксис Markdown для создания структурированных документов с заголовками, списками, таблицами, изображениями и блоками кода. Изучил процесс компиляции Markdown-документов в форматы PDF и DOCX с использованием Pandoc и Makefile.

Список литературы